

CHAPTER 2 — STRING INPUT & OUTPUT

SLIDE 1 — READING A STRING

LET'S GET INPUT!

In Chapter 1, we learned how to **create and store a string**.

But what if the user wants to enter their own name? 

We need to **read the string from the user**.

Simple Definition

Reading a string means taking text entered by the user and storing it in a string variable.

Example

```
char name[20];
```

```
scanf("%s", name);
```

WHAT IS HAPPENING?

 USER



 Types "Ravi"



 C READS THE INPUT



 name



"Ravi"

EASY ANALOGY

Think of your program as a **form** .

 NAME: _____

 You type → Ravi

 Program stores → "Ravi"

★ **REMEMBER!**

 **Input = Information given to the program**

 **Reading = Taking that information into the program**

SLIDE 2 — **DISPLAYING A STRING**

★ **NOW LET'S SHOW IT!**

We learned how to **read** a string.

Now we need to **display** it on the screen. 

 **Simple Definition**

Displaying a string means showing the stored text on the screen.

For this, we use:

`printf()`

 **Example**

```
char name[] = "Ravi";
```

```
printf("%s", name);
```

 **OUTPUT**

Ravi

FOLLOW THE JOURNEY

 name
↓
"Ravi"
↓
 printf()
↓
 Screen
↓
Ravi

EASY ANALOGY

Think of `printf()` as a **speaker** .

 STRING
↓
 printf()
↓
 "Ravi"

It takes the stored information and **shows** it.

★ REMEMBER!

 `printf()` → "C, show this!"

SLIDE 3 — USING `scanf()`

WHAT IS `scanf()`?

`scanf()` is a C function used to **read input from the user**.

For a string, we commonly use:

`%s`

Example

```
char name[20];
```

```
scanf("%s", name);
```

BREAK IT DOWN

scanf() →  Read input

%s →  String

name →  Where the string is stored

SEE IT IN ACTION

If the user enters:

Ravi

Then:

 Ravi



 scanf("%s", name)



 name



"Ravi"

COMPLETE MINI PROGRAM

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    char name[20];
```

```
    printf("Enter your name: ");
```

```
    scanf("%s", name);
```

```
    return 0;  
}
```

MEMORY TRICK

 `scanf()` → "C, take my input!"

QUICK NOTE

With `%s`, `scanf()` reads a **word** rather than a complete line containing spaces.

SLIDE 4 — USING `printf()`

WHAT DOES `printf()` DO?

`printf()` is used to **display the string stored in a variable**.

For strings, we use:

`%s`

Example

```
char name[] = "Ravi";
```

```
printf("%s", name);
```

OUTPUT

Ravi

BREAK IT DOWN

`printf()`



 Display

`%s`



 String

name



 String we want to display

 **THINK OF IT LIKE THIS**

 name

"Ravi"



 printf("%s", name)



 Ravi

 **ANOTHER EXAMPLE**

```
char city[] = "Chennai";
```

```
printf("I live in %s", city);
```

 **OUTPUT**

I live in Chennai

★ **REMEMBER!**

%s → Tells **printf()** that the value is a string 

printf() → Displays the string 

SLIDE 5 — USING **fgets()** + **RECAP**

★ **ANOTHER WAY TO READ A STRING**

We can also use:

fgets()

Simple Definition

fgets() is used to read a string from the input.

Example

```
char name[20];
```

```
fgets(name, 20, stdin);
```

WHAT IS HAPPENING?

 USER



 Types input



 fgets()



 name



 String stored

vs **scanf()** & **fgets()**

 scanf()



Reads a string

 fgets()



Reads a string/line

For example, **fgets()** can read input containing spaces:

Hello World

⚡ STRING INPUT & OUTPUT — QUICK RECAP!

🧠 REMEMBER THESE!

🖥️ Reading a String

→ Take text from the user.

📦 `scanf()`

→ Reads string input.

💡 `printf()`

→ Displays a string.

📝 `%s`

→ Used for strings with `scanf()` / `printf()`.

📖 `fgets()`

→ Reads a string/line of input.

🎯 ONE BIG MEMORY TRICK

👤 USER



🖥️ INPUT



📦 `scanf()` / 📖 `fgets()`



📦 STRING



💡 `printf()`



🖥️ OUTPUT

🏆 LEVEL CLEARED!

I can now read and display strings in C! 🎉

QUESTIONS:

Q1. MCQ — Displaying a String

Which function is mainly used to display a string on the screen? 

- ♦ A) `scanf()`
- ♦ B) `printf()`
- ♦ C) `fgets()`
- ♦ D) `strlen()`

 Hint: Think about the function that sends information to the screen.

 Glossary:

-  Display — Show something on the screen.

 Answer: B) `printf()`

Q2. MCQ — Reading a String

Which format specifier is used with `scanf()` to read a string? 

- ♦ A) `%d`
- ♦ B) `%c`
- ♦ C) `%s`
- ♦ D) `%f`

 Hint: `%s` is connected with a string.

 Glossary:

-  Format specifier — A symbol that tells C what type of data is being handled.

 Answer: C) `%s`

Q3. Fill in the Blank — `scanf()`

Complete the code to read the user's name.

```
#include <stdio.h>
```

```
int main()
{
    char name[20];

    printf("Enter your name: ");
    _____("%s", name);

    return 0;
}
```

 Hint: Which function is used to take input from the user?

 Glossary:

-  Input — Information given by the user.
-  `scanf()` — Reads input from the user.

 Answer: `scanf()`

Q4. Fill in the Code — `printf()`

Complete the code to display the stored string.

```
#include <stdio.h>
```

```
int main()
{
    char name[] = "Ravi";

    _____("%s", name);

    return 0;
}
```

```
}
```

- ♦ A) `scanf`
- ♦ B) `fgets`
- ♦ C) `printf`
- ♦ D) `strlen`

💡 Hint: The string is already stored. Now the program needs to show it. 🙄

📖 Glossary:

- 💾 Stored — Kept in a variable.
- 🖥 Display — Show on the screen.

✅ Answer: C) `printf`

📖 Q5. Fill in the Code — `fgets()`

Complete the code to read a string using `fgets()`.

```
#include <stdio.h>
```

```
int main()
{
    char message[50];

    _____;

    printf("%s", message);

    return 0;
}
```

- ♦ A) `scanf("%s", message)`
- ♦ B) `fgets(message, 50, stdin)`
- ♦ C) `printf("%s", message)`
- ♦ D) `fgets(message, 50)`

💡 Hint: `fgets()` needs the string variable, size, and input stream. 📦

 Glossary:

-  **fgets()** — Reads a string/line of input.
-  **stdin** — Standard input, usually the keyboard.
-  **Size** — Maximum number of characters to read.

 **Answer: B) fgets(message, 50, stdin)**